Classification de Malwares par Analyse Statique et Apprentissage Supervisé

Projet Machine Learning

27 mars 2026

1 Introduction et Contexte

La prolifération des menaces informatiques sous Windows nécessite des outils de détection capables de traiter des fichiers au format **Portable Executable (PE)**. Face aux limites des signatures traditionnelles, l'approche par *Machine Learning* permet d'identifier des comportements malveillants de manière heuristique. Ce projet vise à concevoir un classifieur intelligent basé sur l'analyse statique, évitant ainsi les risques liés à l'exécution de codes suspects.

2 Problématique

Le défi majeur de ce projet consiste à transformer des caractéristiques structurelles brutes, issues d'un **dataset propriétaire au format Excel**, en descripteurs numériques discriminants. L'enjeu est de déterminer quel algorithme offre le meilleur compromis entre la détection des menaces (Rappel) et la réduction des fausses alertes (Précision).

3 Objectifs du Projet

3.1 Exploitation du Dataset et Prétraitement

Le projet s'appuie sur un jeu de données structuré regroupant les métadonnées de fichiers sains et malveillants. Les étapes clés incluent :

- **Analyse Exploratoire :** Visualisation des données et identification des tendances.
- **Normalisation :** Mise à l'échelle des données pour les algorithmes sensibles aux distances.

3.2 Étude Comparative Multimodèle

Trois architectures algorithmiques seront mises en concurrence :

1. **Support Vector Machine (SVM)** : Pour sa précision dans les espaces de haute dimension.
2. **Random Forest** : Pour sa robustesse face au bruit et aux données tabulaires.
3. **K-Nearest Neighbors (KNN)** : Pour son approche basée sur la proximité géométrique.

3.3 Optimisation du Modèle Champion

Le modèle présentant le meilleur **F1-Score** fera l'objet d'une optimisation avancée via la méthode *GridSearchCV* pour affiner les hyperparamètres.

3.4 Déploiement et Inférence

Mise en œuvre d'une architecture micro-service permettant :

— L'upload d'un binaire (.exe, .dll).
— L'extraction dynamique via la bibliothèque `pefile`.
— La prédiction instantanée par le modèle optimisé via une interface **FastAPI**, **Flask** ou **Streamlit**.

4 Méthodologie et Évaluation

La performance sera évaluée selon la métrique du F1-Score, calculée comme suit :

$$F1 = 2 \cdot \frac{\text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}} \tag{1}$$

5 Environnement Technique

— **Langage :** Python 3.10+
— **Analyse PE :** `pefile`
— **Machine Learning :** *Scikit-learn, Pandas, Joblib*
— **Déploiement :** *FastAPI, Flask ou Streamlit.*